AIMLCZG523 | ASSIGNMENT 01

Heart Disease Prediction

End-to-end machine learning development, experiment tracking, CI/CD, containerization, local Kubernetes deployment, and monitoring.

Report date: 19 October 2026
Repository: github.com/rimjhimdubey73/mlops_assignment
Dataset: UCI Heart Disease, processed Cleveland subset

Executive summary

This submission builds a binary heart-disease classifier from the processed Cleveland dataset. Numeric and categorical preprocessing is fitted inside stratified cross-validation to avoid leakage. Logistic Regression and Random Forest are compared with nested cross-validation; the Logistic Regression pipeline is selected by mean outer-fold ROC-AUC and packaged with MLflow. A FastAPI service serves predictions and Prometheus metrics, logs structured request metadata, and is deployed with Prometheus and Grafana to a local Minikube cluster.

Verified outcome

- 7 tests pass and Ruff reports no issues in the local Python 3.12 environment.
- Docker image built successfully; the model-loaded health endpoint and sample prediction returned HTTP 200 in the VM.
- API, Prometheus, and Grafana Kubernetes deployments rolled out; Prometheus target was up and the provisioned Grafana dashboard was found.
- GitHub Actions screenshot shows a green quality-and-training job with training, container build, smoke test, and artifact upload.

Scope note

This is an educational classifier, not a clinical diagnostic tool. Cross-validation results are estimates on a small processed subset, not evidence of clinical safety or external validity. The local Minikube LoadBalancer has no external IP; access instructions are provided instead of claiming a public production endpoint.

01 | DATA

Data acquisition and preparation

Source and target

The data acquisition module downloads the processed Cleveland file from the UCI Heart Disease repository and validates its 14-column schema. The original target `num` is an ordinal severity value from 0 to 4. For the assignment's binary classification problem, values 0 are mapped to no disease and values 1 through 4 to disease present.

Cleaning decisions

- The raw file uses `?` for unknown measurements; these values are converted to missing values.
- Continuous measurements are median-imputed; categorical measurements are mode-imputed.
- Categorical predictors are one-hot encoded. The EDA-only processed CSV has a binary `target` and 28 encoded predictor columns.
- The source contains 303 rows, 4 missing `ca` values, and 2 missing `thal` values. Duplicate rows were checked; none were found.

Reproduction

```
python -m src.data_acquisition
python -m src.data_preprocessing
```

The raw source is retained separately from the EDA-imputed table. Model training reads the raw features and places all imputation, scaling, and encoding inside sklearn pipelines fitted independently on each training fold.

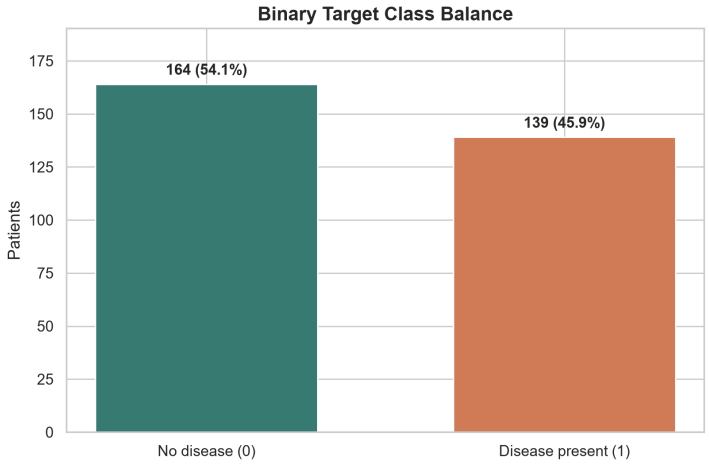
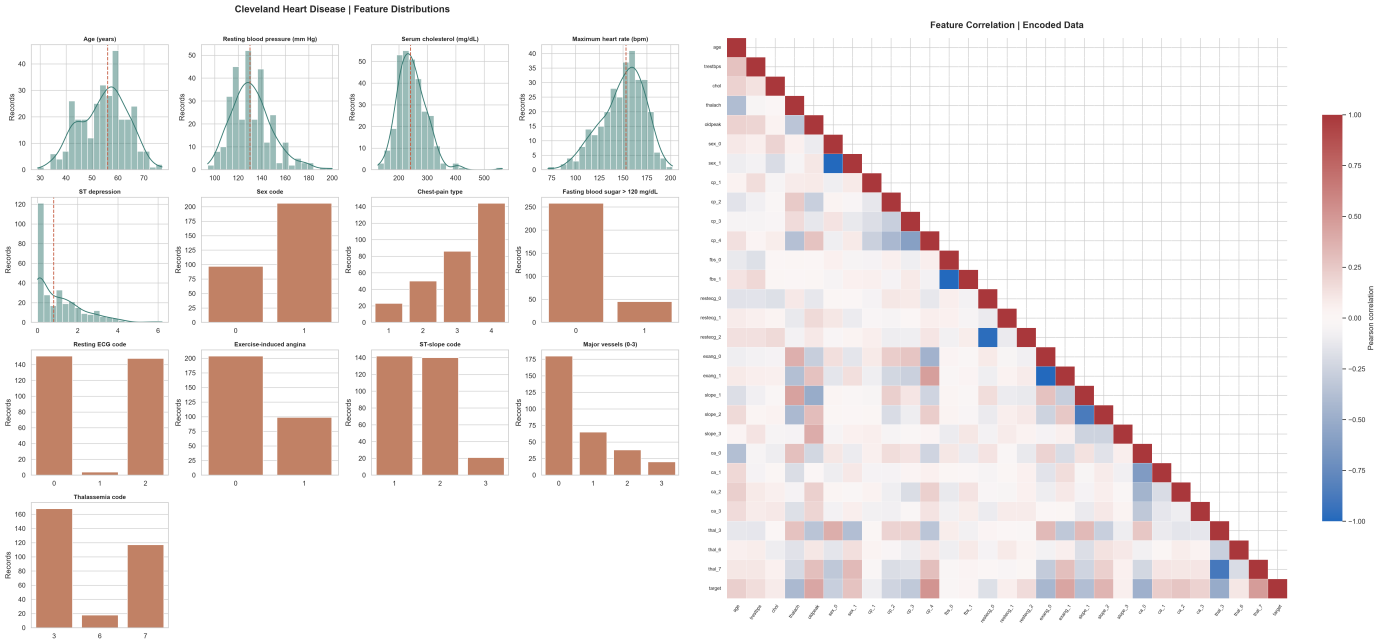


Figure 1. Binary class counts after mapping severity to disease absent/present.

01 | EDA

Exploratory analysis

The notebook audits missingness and class balance, plots the numeric feature distributions, and creates a correlation heatmap from the prepared predictors. EDA imputations are used only to make descriptive figures; the modeling pipeline refits preprocessing in each cross-validation fold to prevent leakage.



Figures 2-3. Numeric distributions (left) and encoded-feature correlation heatmap (right).

Interpretation and limits

The target classes are moderately balanced: 164 examples (54.13%) without disease and 139 (45.87%) with disease. Age, chest-pain category, maximum heart rate, exercise-induced angina, and ST depression are clinically interpretable inputs in the dataset, but visual association is not causation. The modest sample size and single processed regional subset limit generalization.

02 | MODELING

Feature engineering and model development

Pipeline design

Each candidate is a single sklearn Pipeline with a ColumnTransformer. Numeric features receive median imputation followed by StandardScaler. Categorical features receive most-frequent imputation and OneHotEncoder with unknown categories ignored at inference. Because preprocessing is inside the estimator, validation folds do not learn imputation values, scaling statistics, or category vocabularies from held-out data.

Tuning procedure

- Nested stratified cross-validation: 4-fold inner search optimizes ROC-AUC; 5-fold outer CV estimates generalization.
- Logistic Regression searches C in {0.1, 1, 10} and unweighted/balanced class weights.
- Random Forest uses 200 trees and searches depth, minimum leaf size, and unweighted/balanced class weights.
- Accuracy, precision, recall, and ROC-AUC are reported as outer-fold mean +/- sample standard deviation.

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.858 +/- 0.056	0.864 +/- 0.060	0.820 +/- 0.084	0.920 +/- 0.024
Random Forest	0.828 +/- 0.018	0.823 +/- 0.059	0.806 +/- 0.055	0.916 +/- 0.023

Selection

Logistic Regression is selected by mean outer-fold ROC-AUC (0.9195 versus 0.9156). The difference is small relative to fold variability, so the selection is a pragmatic model-selection result, not evidence of a statistically significant advantage. The reported scores are cross-validation estimates; there is no independent external test set.

```
python -m src.model_development
```

03-04 | MLFLOW AND REPRODUCIBILITY

Experiment tracking and model package

MLflow experiment

Training creates one MLflow run for each classifier in the `heart-disease-model-development` experiment using a local SQLite tracking database and file artifact store. Runs record model family, random state, fold configuration, raw dataset name, training-row count, tuned parameters, and metric means and standard deviations. Logged artifacts include the comparison CSV, inner-search details and outer-fold parameter choices, EDA figures when available, and metric plots. The winning run is tagged and includes the logged model URI.

Packaged inference pipeline

The selected full-data pipeline is logged as an MLflow sklearn model and saved to the versioned `models/` directory. The package carries fitted preprocessing and classifier objects, an input example, model signature, pinned dependency requirements, and a `current_model.txt` pointer. It uses skops serialization with an explicit trusted-type allowlist. Serving accepts the original 13 raw features so preprocessing is identical to training.

```
mlflow ui --backend-store-uri sqlite:///mlflow.db
# Open http://127.0.0.1:5000

cat models/current_model.txt
```

Reproducibility

- Use the declared `requirements.txt` and `requirements-dev.txt` in a clean Python environment.
- Acquire source data with the script; do not rely on a manually edited or pre-imputed training table.
- Nested CV and model search use a fixed random seed and explicit search grids.
- The versioned model package used for serving is produced by the training command and is a small local artifact; retrain to recreate it.

The experiment database, logs, and model output are generated artifacts and are not all committed. The repository's CI workflow uploads them on each successful run.

05 | QUALITY GATES

Automated tests and CI/CD

Local quality checks

```
python -m pip install -r requirements-dev.txt
ruff check src tests app.py
python -m pytest -q
```

The current local verification passed: Ruff clean; 7 pytest tests passed. Tests cover data missing-value handling and target conversion, the model pipelines' fit/predict behavior including missing/unseen categories, API schema behavior, metrics exposition, and request logging. One upstream Starlette deprecation warning remains and does not fail the suite.

Hosted workflow

The GitHub Actions workflow runs on pushes, pull requests, and manual dispatch. It installs dependencies, lints, tests, downloads data, trains and tracks models, builds the serving Docker image, starts it, exercises health/prediction/metrics endpoints, and uploads model, comparison, MLflow, and screenshot artifacts. Shell failure handling collects API container logs when a smoke test fails.

Evidence

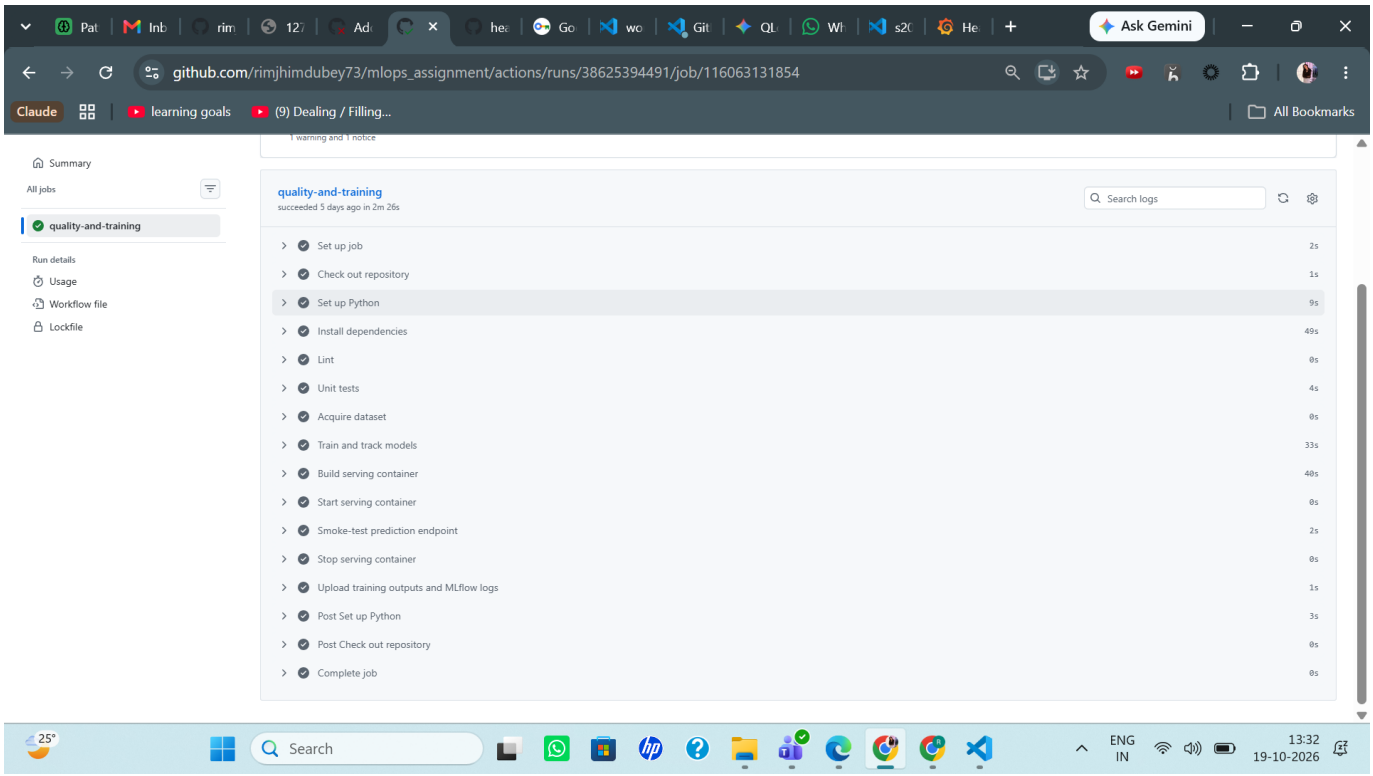


Figure 4. Uploaded GitHub Actions job log: quality-and-training completed successfully, including the container build and smoke test.

The uploaded run screenshot is genuine hosted CI evidence. The screenshot is useful for demonstrating this run, but future source changes should be followed by another successful run before submission.

06 | SERVING

API and container verification

FastAPI service contract

- `GET /health` reports service status and whether the model loaded.
- `POST /predict` requires all 13 raw feature keys; values can be null for missing measurements.
- Prediction response includes binary class, human-readable label, predicted-class confidence, and both class probabilities.
- `GET /metrics` exposes Prometheus request and latency metrics; JSON logs include request ID, method, path, status, and duration without logging request bodies.

Build and run

```
docker build -t heart-disease-api:latest .
docker run --rm -p 8000:8000 heart-disease-api:latest

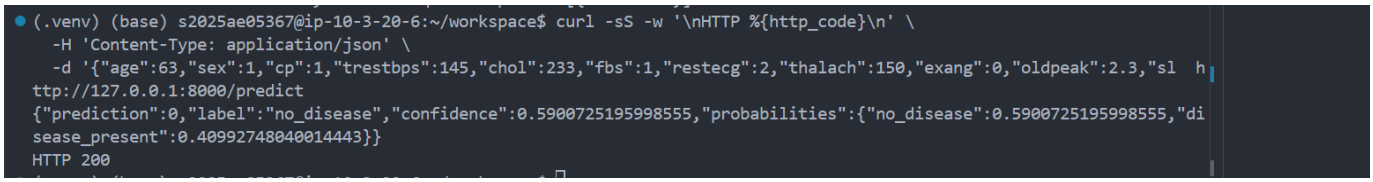
curl http://127.0.0.1:8000/health
```

Container verification

The Docker image built successfully from the trained package. The documented sample payload returned HTTP 200 with prediction 0 (`no_disease`), confidence 0.5901, and probabilities 0.5901 / 0.4099. The same behavior was verified in the Minikube deployment. A public Dev Tunnel request previously returned 504 during prediction while direct VM requests returned 200; this is recorded as a tunnel limitation, not an API test failure.

Successful prediction evidence

The executed sample request, HTTP 200 status, prediction, confidence, and both probabilities are shown in `reports/screenshots/03-predict-endpoint.png`. The terminal screenshot was cropped from the original uploaded capture, which is retained in the screenshot folder.



07 | LOCAL PRODUCTION DEPLOYMENT

Kubernetes deployment

The image is deployed to Minikube 1.35.0 running Kubernetes 1.32.0. The namespace contains a one-replica FastAPI deployment, a LoadBalancer Service, Prometheus, and Grafana. API probes, non-root execution, read-only root filesystem, resource requests/limits, and ephemeral writable volumes are configured. Prometheus and Grafana are internal ClusterIP services.

Verified rollout

- API, Prometheus, and Grafana deployments each reached one ready replica after Minikube was restarted.
- Health and sample prediction returned HTTP 200 through the VM port-forward.
- The API Service external IP is ``; no public cloud endpoint is claimed.

```
minikube start --driver=docker --cpus=2 --memory=4096
minikube kubectl -- rollout status deployment/heart-disease-api -n heart-disease
minikube kubectl -- get pods,services -n heart-disease -o wide
minikube kubectl -- port-forward -n heart-disease service/heart-disease-api 8000:80
```

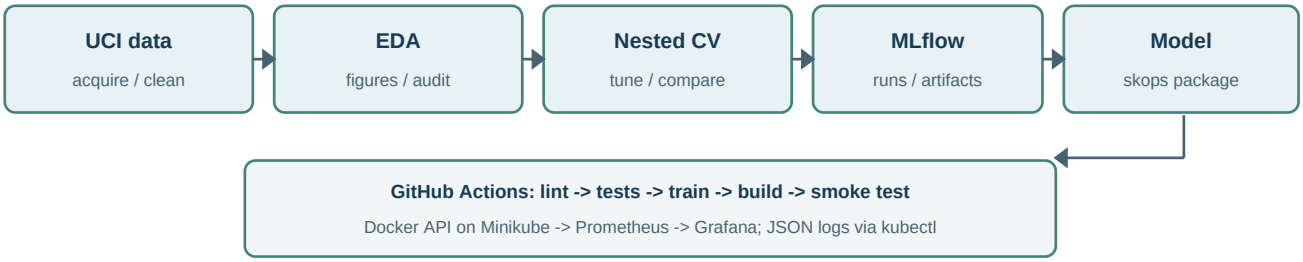


Figure 5. Data-to-serving architecture and automated CI path.

The live terminal workload capture is not yet in the screenshot folder. The commands above reproduce it. Minikube can stop when the VM is idle; run `minikube start` before capturing deployment output or using the API.

08 | OBSERVABILITY

Monitoring and request logging

Prometheus scrapes the API `/metrics` endpoint every 15 seconds. The provisioned Grafana dashboard includes request rate, 95th percentile latency, status-class rates, and target health. The API emits single-line JSON access/error logs to stdout; Kubernetes users can inspect them with `minikube kubectl -- logs -n heart-disease deployment/heart-disease-api`.

Verified monitoring

- Prometheus API target `heart-disease-api` reported health `up`.
- Grafana 12.1.1 health endpoint returned database status `ok`.
- The `Heart Disease API` dashboard was found in Grafana's provisioned dashboard list.
- Monitoring uses emptyDir storage for this local demonstration; metrics history is not durable after pod replacement.

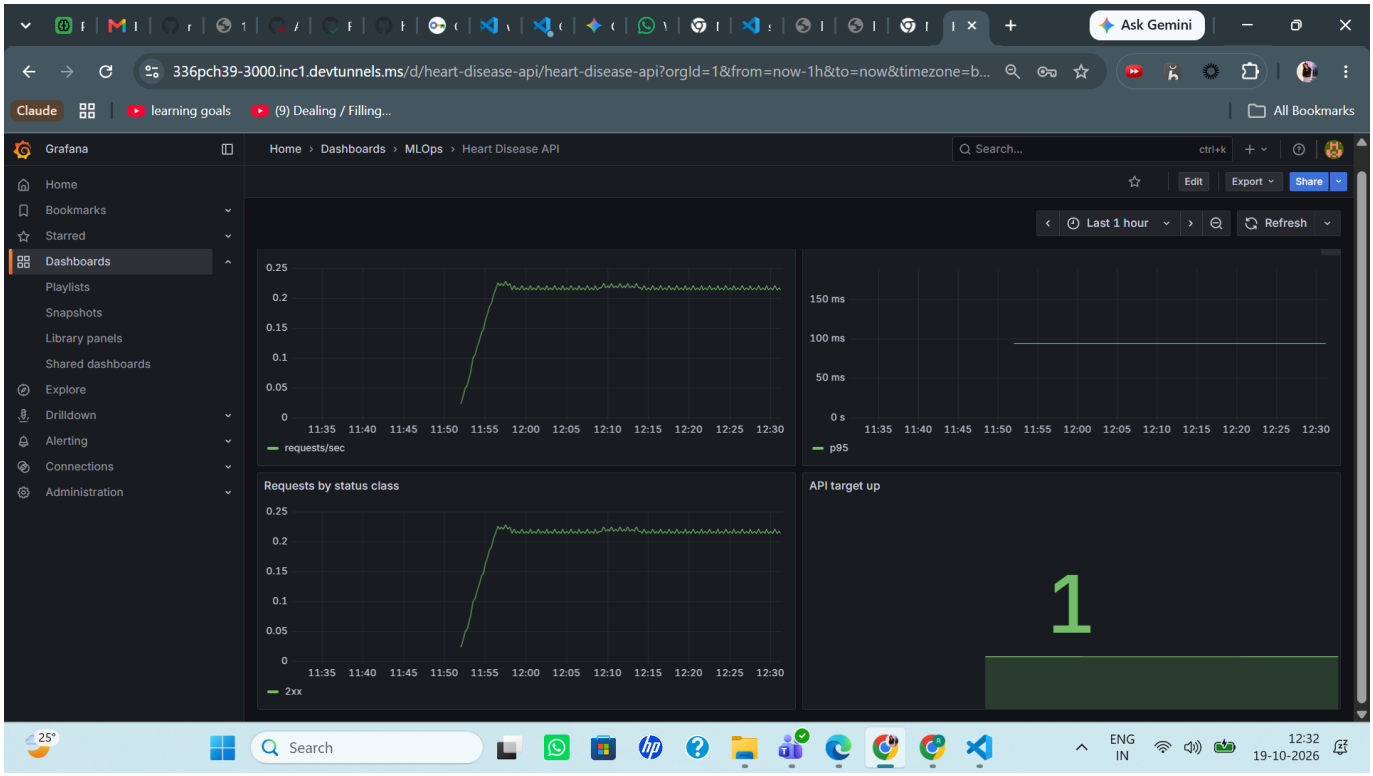


Figure 6. Uploaded Grafana dashboard capture. Review the visible panel data and recapture if the panels do not show recent API traffic.

```
minikube kubectl -- port-forward -n heart-disease service/prometheus 9090:9090
minikube kubectl -- port-forward -n heart-disease service/grafana 3000:3000
```

The Grafana admin password is stored in a Kubernetes Secret and must not be committed to the repository. Use the secret in the live cluster to sign in; rotate it before leaving the demonstration environment if it has been shared.

09 | CHECKLIST

Submission readiness and next actions

Completed and verified

- UCI data acquisition, cleaning, EDA figures, preprocessing, two candidate models, nested CV, and documented metrics.
- MLflow experiments, logged artifacts, selected serialized pipeline, requirements, and inference API.
- Unit tests and Ruff; passing hosted GitHub Actions screenshot; Docker image build and smoke test.
- Minikube API/monitoring deployments, endpoint verification, Prometheus scrape, Grafana dashboard, and request logs.
- Repository link: https://github.com/rimjhimdubey73/mlops_assignment.

Still required from the submitter

- Capture and name `01-kubernetes-workloads.png`, `02-health-endpoint.png`, and `03-predict-endpoint.png` from live successful outputs. Existing Swagger example screens are not substitutes for executed requests.
- Commit or upload the generated `models/` package if the grader must build the image without running training first; otherwise explain that training creates it.
- Use the README's port-forward instructions to access the deployed Minikube API. Keep the Grafana password out of Git.

Academic integrity and scope

Be prepared to explain target binarization, leakage prevention, nested CV, why ROC-AUC selected the final candidate, how the packaged preprocessing travels with the estimator, what the API confidence means, and the limitations of a small regional dataset. Do not represent screenshots or results as your own experimentation unless you can explain and reproduce them.

Full setup commands are in `README.md`; the report is generated with `python reports/build_final_report.py` after installing `requirements-dev.txt`.